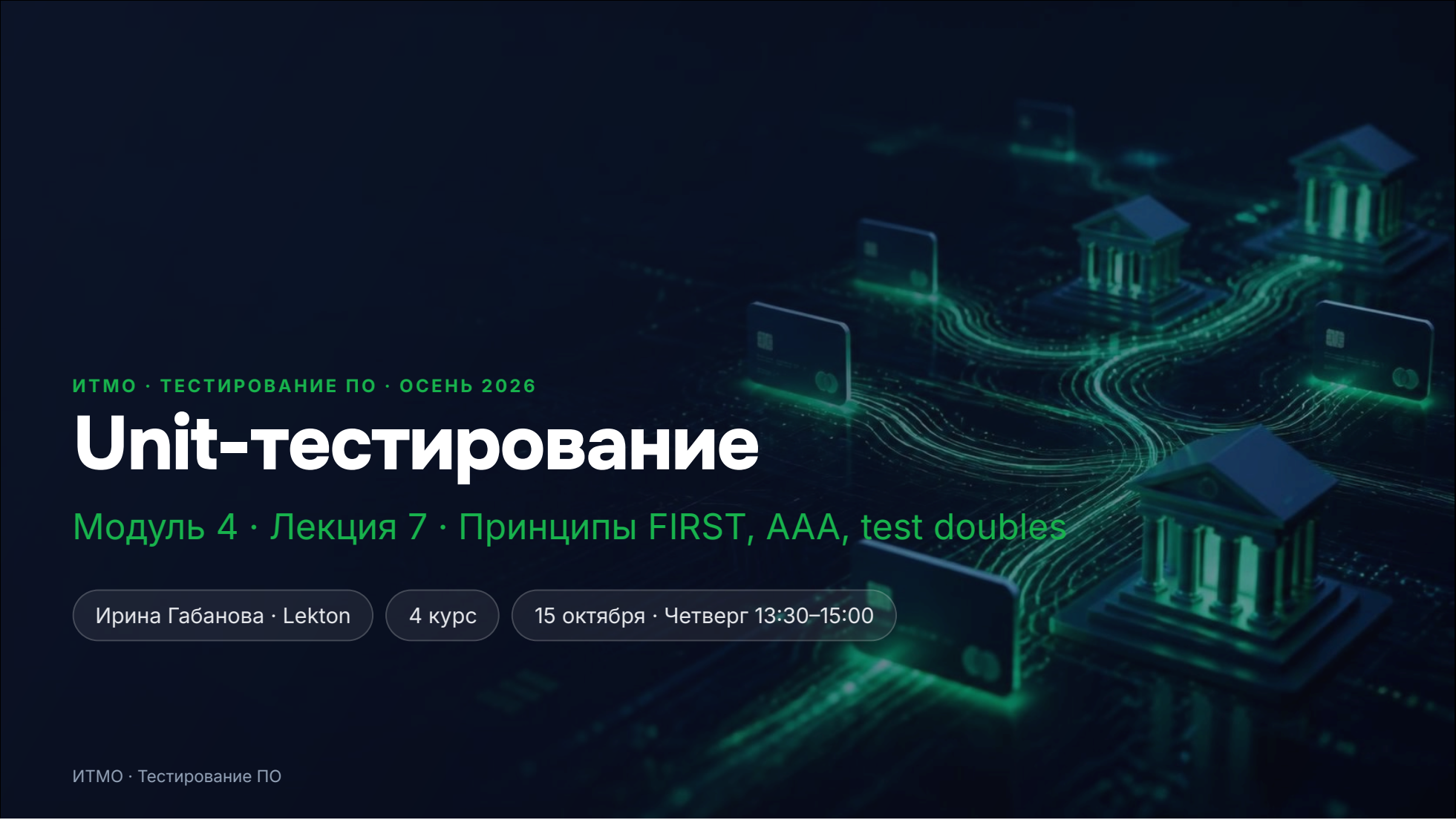


ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Unit-тестирование

Модуль 4 · Лекция 7 · Принципы FIRST, AAA, test doubles

Ирина Габанова · Lekton

4 курс

15 октября · Четверг 13:30–15:00

МОДУЛЬ 4 · ЛЕКЦИЯ 7

План лекции

01 Зачем нужны unit-тесты: быстрая обратная связь, документация, цель — устойчивый рост

02 Что такое unit-тест: три атрибута

03 Структура теста AAA и принципы FIRST

04 Test Doubles: Stub, Mock, Fake и главное правило CQS

05 Инструменты: JUnit 5, Mockito, AssertJ

06 Интерактив: «Найди дефект теста» + викторина «Stub или Mock?»

МОДУЛЬ 4 · ЛЕКЦИЯ 7

Зачем нужны unit-тесты

Быстрая обратная связь. Секунды вместо минут и часов: изменил строку — сразу знаешь, не сломал ли что-то.

Живая документация. Тест не может устареть молча: если врёт о поведении — падает.

Безопасный рефакторинг. Тесты «кричат», если поведение изменилось. Код без тестов менять страшно.

Дешевле чинить дефекты. Баг пойман в момент написания кода, а не в проде.

Экономика итераций: цена итерации обратной связи определяет, сколько итераций вы можете себе позволить. Секунды — тысячи прогонов в день. Часы — три-четыре.

Честная цена (Хориков): тесты — тоже код, они стоят денег. Цель — не «больше тестов», а **больше ценных тестов**.

Цель unit-тестирования: устойчивый рост

Цель — не «покрытие» и не «зелёная полоса». Unit-тестирование — техника, позволяющая проекту **расти устойчиво**: наращивать функциональность и рефакторить код без деградации качества (Khorikov, гл. 1).

Тесты — страховка. Позволяют менять код без страха «сломать незаметно». Нет тестов → ручной регресс за дни → «работает — не трогай» → код гниёт.

Тесты — исполняемая спецификация. Читая тесты класса, новичок за 10 минут понимает, как он должен себя вести. Документация врёт молча, тесты падают.

Антицель: тесты, привязанные к реализации. Ломаются при каждом рефакторинге, даже когда поведение не изменилось. Такие тесты — не страховка, а балласт.

Перекличка с модулем 1: там мы разбирали график стоимости дефекта — стоимость исправления растёт с уровнем (требования → unit → интеграция → E2E → прод). Unit-тесты — самая ранняя и самая дешёвая точка поимки. Здесь важна не цифра, а принцип: *чем раньше, тем дешевле*.

Девиз модуля: тесты должны проверять **поведение**, а не устройство кода. Почему это важно и как достичь — тема правила CQS и следующей лекции.

Что такое unit-тест: три атрибута

Маленький кусок кода. Единица — не метод, а **единица поведения**: один сценарий. У метода `authorize` — отдельный тест на одобрение, на заблокированную карту, на отрицательную сумму.

Быстро. Миллисекунды. Быстрые тесты запускают после каждого изменения — дефект ловится по цене 1x.

Изолированно. Зелёный поодиночке, зелёный всем набором, в любом порядке, на любой машине.

ISTQB-аналог: **component testing**, самый низкий уровень тестирования (модуль 3).

Правило курса: мокируем только то, что мешает скорости и воспроизводимости — сеть, БД, время. Отсюда главное правило CQS на второй половине лекции.

МОДУЛЬ 4 · ЛЕКЦИЯ 7

Фундамент: структура AAA и принципы FIRST

Структура теста: Arrange — Act — Assert

Паттерн популяризировал Билл Уэйк, детально разобрал Хориков. `blockedCard()` и `request(...)` — приватные хелперы *тест-класса* (Object Mother / Test Data Builder), а не production-код: без них Arrange разросся бы до десятка сеттеров.

```
@Test
void shouldDeclineWhenCardIsBlocked() {
    // Arrange — приводим систему в нужное состояние
    CardRepository repo = mock(CardRepository.class);
    when(repo.findByPan(PAN)).thenReturn(blockedCard());
    AuthService service = new AuthService(repo);

    // Act — ровно один вызов тестируемого метода
    AuthResult result = service.authorize(request(PAN, 1000));

    // Assert — одна логическая проверка решения
    assertThat(result.getStatus()).isEqualTo(DECLINED);
    assertThat(result.getReason()).isEqualTo(CARD_BLOCKED);
}
```

Один тест — одно действие. Два вызова в Act — два сценария; при падении не понять, какой сломан.

Две проверки на один Act — допустимо: статус + причина = одна история об отказе.

Имя теста — спецификация: should...When... читается как требование. Альтернатива AAA — Given-When-Then (BDD).

Принципы FIRST

F

ast — быстрые: миллисекунды. Нарушение: реальная БД, сеть, `Thread.sleep`.

I

solated — независимы друг от друга и от порядка. Нарушение: общее состояние, каскад ложных падений.

R

epeatable — одинаковый результат везде. Нарушение: время, случайность, локаль, внешние сервисы.

S

elf-validating — зелёный/красный без ручного анализа. Нарушение: «открой лог и посмотри глазами».

T

imely — пишутся вместе с кодом (или до — TDD). Нарушение: «потом, когда будет время».

Fast — это экономика: секунды — прогон после каждого изменения; минуты — раз в час; десятки минут — «археология» раз в день.

Repeatable: время и случайность подменяем зависимостями (Clock, фиксированный seed) — это же улучшает дизайн.

Timely и TDD: тест до кода (Red → Green → Refactor) даёт два бонуса — тесты точно написаны, а код под тестом получается тестируемым и лучше спроектированным.

Первый тест целиком: Money

```
class MoneyTest {
    @Test
    void shouldAddSameCurrency() {
        Money five = Money.dollars(5);

        Money result = five.plus(Money.dollars(3));

        assertThat(result).isEqualTo(Money.dollars(8));
    }
}
```

Fast: чистая арифметика в памяти — микросекунды.

Isolated: свои объекты, ничего не оставил после себя.

Repeatable: нет времени, случайности, внешних систем.

Self-validating: один assert решит зелёный/красный.

Timely: у Бека написан до реализации `plus` — TDD.

AAA-пропорции: одна строка подготовки, одна — действия, одна — проверки. В реальных тестах Arrange растёт, Act и Assert почти всегда остаются короткими.

Поведение, а не внутренности: сравнение через `equals` значимого объекта, не через геттеры полей. Тесты, привязанные к реализации, ломаются при рефакторинге (об этом — лекция 8).

МОДУЛЬ 4 · ЛЕКЦИЯ 7

Test Doubles: заглушки и моки

МОДУЛЬ 4 · ЛЕКЦИЯ 7

Test Doubles: классификация Месароша

Проблема: у кода есть зависимости (БД, сеть, часы, другие сервисы). Реальная база нарушит *Fast*, сеть — *Repeatable*, общее состояние — *Isolated*. Решение — подменить зависимость дублёром: как каскадёр в кино — на экране неотличим, но работает в опасных сценах.

Dummy
Заполнитель параметра, который в сценарии не используется.

Stub
Отдаёт готовые ответы на вызовы. Ничего не проверяет.

Spy
Stub + ведёт протокол: какие методы, с какими аргументами, сколько раз.

Mock
Заранее заданные ожидания вызовов; сам проверяет их через verify.

Fake
Рабочая упрощённая реализация: in-memory репозиторий вместо PostgreSQL.

Почему важна точность терминов: в индустрии всё называют словом «мок» — из-за этого путаница и неправильное использование. От типа дублёра зависит, что и как проверять. Testcontainers и Hoverfly (модуль 6) — по сути фейки нового поколения.

Stub vs Mock: принцип CQS

Запрос (query) → Stub

Возвращает значение, не меняет состояние.
`findByPan(pan)` — запрос.
Результат проверяем **через Assert** теста.
Если сервис не спросил карту — он не примет решение, тест упадёт сам, без verify.

Команда (command) → Mock

Меняет состояние, ничего не возвращает.
`notificationClient.send(n)` — команда.
Факт вызова проверяем **через verify**.
Без verify вы никак не узнаете, что уведомление отправлено.

НИКОГДА не verify вызовы stub'а — это overspecification. Сценарий: разработчик добавил кэш карт — сервис перестал ходить в репозиторий при повторном запросе. Поведение для клиента не изменилось, а тест покраснел. Ложные падения подрывают доверие: их игнорируют, потом отключают, потом набор тестов мёртв.

Правило одной строкой: verify — только для команд с наблюдаемыми побочными эффектами наружу. Всё остальное — через конечный результат.

От Фримена и Прайса: мокируйте только типы, которыми владеете. Чужую библиотеку оборачивайте в тонкий адаптер своего кода — и мокируйте адаптер.

Mockito: минимальный набор

`@Mock` — создать мок-объект

`@InjectMocks` — собрать объект с моками

`when / thenReturn` — настройка stub'а

```
@ExtendWith(MockitoExtension.class)
class AuthServiceTest {
    @Mock CardRepository cardRepository;
    @Mock NotificationClient notificationClient;
    @InjectMocks AuthService service;

    @Test
    void shouldNotifyOnDecline() {
        when(cardRepository.findByPan(PAN)).thenReturn(blockedCard()); // query → stub

        service.authorize(request(PAN, 1000));

        verify(notificationClient).send(argThat(n → n.getPan().equals(PAN))); // command → mock
    }
}
```

CQS на практике: `when…thenReturn` на запросе — и никакого verify. `verify` — только на команде `send`.

Осторожно с `@InjectMocks`: в реальном коде надёжнее явный `new AuthService(...)` в `@BeforeEach` — ошибка компиляции вместо загадочного NullPointerException.

МОДУЛЬ 4 · ЛЕКЦИЯ 7

Инструменты: JUnit 5, Mockito, AssertJ

JUnit 5: жизненный цикл

```
@BeforeAll  static void initAll()     // один раз до всех тестов (static!)
@BeforeEach void init()               // перед КАЖДЫМ тестом
@Test       void test1()              // тест
@AfterEach  void tearDown()           // после КАЖДОГО теста
@AfterAll   static void tearDownAll() // один раз после всех (static!)
```

Ключевой факт: JUnit создаёт **новый экземпляр тестового класса** на каждый `@Test`. Поля экземпляра физически не могут протечь из одного теста в другой — фреймворк помогает соблюдать принцип Isolated.

@BeforeAll / @AfterAll обязаны быть static — экземпляр пересоздаётся, поля экземпляра им недоступны. В unit-тестах нужны редко.

Здоровый @BeforeEach: каждый тест класса использует почти всё, что тот создаёт. Иначе — разнести классы или фабричные методы.

Ещё в арсенале: `@Disabled("причина")`, `@Timeout(1)` — защита от слипов и дедлоков, `@Nested` — группировка по сценариям (лекция 8).



Параметризованные тесты

Один метод-тест, много наборов данных. JUnit прогонит метод для каждой строки и **отчитается по каждой отдельно** — упадёт только нарушенная строка, вы сразу видите, какой класс эквивалентности сломан.

```
@ParameterizedTest(name = "{0} + {1} ⇒ {2}")
@CsvSource({
    "100,  OK,      APPROVED",   // внутри допустимого класса
    "0,    OK,      DECLINED",   // граница: нулевая транзакция?
    "-50,  OK,      DECLINED",   // вне допустимого класса
    "100,  BLOCKED, DECLINED"    // решает статус карты, не сумма
})
void shouldAuthorizeByAmountAndStatus(long amount, CardStatus status, Decision expected) {
    // один Arrange-Act-Assert на все наборы данных
}
```

Это модуль 2 в коде: каждая строка — классы эквивалентности и граничные значения, превращённые в unit-тесты.

Источники данных: `@ValueSource` — список значений; `@CsvSource` — табличка «входы → ожидание»; `@MethodSource` — сложные данные, собранные кодом.

Жёсткое правило: ветвлений и циклов в теле теста быть не должно. Тест — линейный сценарий от Arrange к Assert; логика в тесте — код, который сам никем не протестирован.

AssertJ: читаемые проверки

```
// JUnit — ожидаемое ПЕРВЫМ, легко перепутать
assertEquals(DECLINED, result.getStatus());

// AssertJ — fluent API, читается как предложение
assertThat(result.getStatus()).isEqualTo(DECLINED);
assertThat(result.getErrors())
    .hasSize(2)
    .containsExactly(CARD_BLOCKED, LIMIT_EXCEEDED);
assertThatThrownBy(() → service.authorize(null))
    .isInstanceOf(InvalidRequestException.class)
    .hasMessageContaining("pan");
```

Порядок перепутать невозможно: фактическое значение — в `assertThat`, дальше цепочка автодополняется IDE. Для коллекций, строк и исключений — типизированные проверки.

Хорошие diff'ы при падении: позиция расхождения в строках, каких элементов не хватает, какие поля объектов различаются. Экономит минуты на каждом красном тесте.

Одна логическая проверка на тест: размер + содержимое одного результата — одна история. Статус + запись в БД + уведомление — три теста. «Если в истории есть слово "и ещё" — это две истории».

Лучшие практики: чек-лист

✅ **Один тест — одна проверка поведения.** Признак нарушения — слово «и» в описании.

✅ **Имя теста = спецификация** (`should…When…`). `test1` заставляет читать код — документации нет. Имена длинными бывают — это нормально.

🚫 **Не тестировать приватные методы напрямую.** Сложный приватный метод — сигнал дизайна: он хочет стать публичным методом отдельного класса.

🚫 **Не дублировать логику продакшна в тесте.** Ожидаемое значение считает «той же формулой»? Тест проверяет «код совпадает с копией кода» — с тем же багом.

✅ **Тест — это код:** ревью, рефакторинг, осмысленные имена. Но небольшое дублирование лучше хитрой абстракции — тест читается сам по себе.

🚫 **Никаких if/loop в тестах** — линейный сценарий. Плюс: красный тест разбираем, не отключаем. Отключённый красный — заглушённая пожарная сигнализация.

Интерактив: «Найди дефект теста»

[ИНТЕРАКТИВ — 5 мин] В каждом тесте спрятан дефект. Найдите проблему и назовите нарушенный принцип: **FIRST / AAA / CQS**. Бонус: как починить?

```
// Тест A
@Test void testAuthorize() {
    service.authorize(request(PAN, 1000));
    service.authorize(request(PAN, -5));
    assertThat(service.getHistory()).isNotEmpty();
}
```

```
// Тест B
@Test void shouldDeclineWhenCardIsBlocked() {
    when(cardRepository.findByPan(PAN))
        .thenReturn(blockedCard());
    AuthResult result =
        service.authorize(request(PAN, 1000));
    verify(cardRepository).findByPan(PAN);
    assertThat(result.getStatus())
        .isEqualTo(DECLINED);
}
```

```
// Тест C
static final List<AuthResult> RESULTS = new ArrayList<>();

@Test void shouldApproveValidTransaction() throws Exception {
    Thread.sleep(500);
    RESULTS.add(service.authorize(request(PAN, 1000)));
    assertThat(RESULTS).isNotEmpty();
}
```

Проверим себя: «Stub или Mock?»

ИНТЕРАКТИВ · «Stub или Mock?» — и нужен ли verify?

- `cardRepository.findByPan(pan)` → ?
- `notificationClient.send(notification)` → ?
- `currencyService.getRate("USD")` → ?
- `auditLog.write(event)` → ?

Ответы — в ходе разбора. Правило одной строкой: **запрос — стаб и assert; команда — мок и verify**.

Выводы и анонс



Unit-тест — код, тестирующий код

Три атрибута: маленький кусок поведения, быстро, изолированно.



Скелет: AAA + FIRST

Arrange-Act-Assert; Fast, Isolated, Repeatable, Self-validating, Timely.



Test Doubles и CQS

Запрос → stub + assert. Команда → mock + verify. Verify на stub — overspecification.



Инструменты

JUnit 5 (новый экземпляр на тест), Mockito, AssertJ, @ParameterizedTest.

Порядок: один тест — одна история об одном факте поведения.

Лекция 8: качество unit-тестов — покрытие, читаемость, сопротивление рефакторингу.

Источники

Khorikov, V. (2020). *Unit Testing: Principles, Practices, and Patterns*. Manning. — Гл. 1–6: четыре столпа теста, AAA, mock vs stub, CQS, стили тестирования.

Beck, K. (2002). *Test Driven Development: By Example*. Addison-Wesley. — Test (noun), Isolated Test, Test First, метафора Programmer's Stone.

Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley. — Таксономия Test Doubles (Dummy/Stub/Spy/Mock/Fake), Object Mother, Test Data Builder.

Freeman, S., & Pryce, N. (2009). *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley. — Tell, Don't Ask; моки; тесты как живая документация; interface discovery.

Fowler, M. (2007). «Mocks Aren't Stubs». *martinfowler.com* — разница между моками и стабами.

Tahchiev, P., Leme, F., Massol, V., & Gregory, G. (2015). *JUnit in Action* (2nd ed.). Manning. — JUnit 5, параметризация, жизненный цикл.

ISTQB — Certified Tester Foundation Level (CTFL) v4.0. — Component testing; экономика дефектов (стоимость растёт с уровнем).


Вопросы?